AI ASSISTED CODING

ASSIGNMENT- 9.1

NAME: E.Shivani

HTNO: 2303A51098

Batch : 02

Problem 1:

Consider the following Python function:

def find_max(numbers):

return max(numbers)

Task:

• Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.

• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

```python
def find_max(numbers):
    '''
    This function returns the maximum value from a list of numbers. It takes a list of numerical values as input and returns the largest numb
    '''
    if not numbers:
        raise ValueError("The input list cannot be empty.") # check if the input list is empty and raise an error if it is
    return max(numbers) # return the maximum value from the list using the built-in max function

def find_max(numbers):
    # Check if the input list is empty and raise an error if it is
    if not numbers:
        raise ValueError("The input list cannot be empty.")
    # Return the maximum value from the list using the built-in max function
    return max(numbers)

def find_max(numbers):
    '''Returns the maximum value from a list of numbers.
    Parameters:
    numbers (list): A list of numerical values.
    Returns:   The maximum value in the list.
    Raises:   ValueError: If the input list is empty.
    '''
    if not numbers:
        raise ValueError("The input list cannot be empty.") # check if the input list is empty and raise an error if it is
    return max(numbers) # return the maximum value from the list using the built-in max function
```

Docstring

- Clear and beginner-friendly.
- Slightly repetitive for a simple function.

Inline comments

- Short and simple.
- Lacks formal documentation.

Google style documentation

- Professional and well-organized.
- Clearly shows parameters, returns, and errors.
- Best for libraries and shared projects.

For a mathematical utilities library, Style 3 is most effective because it provides clear, structured, and reusable documentation.

Problem 2: Consider the following Python function: def login(user, password, credentials):

return credentials.get(user) == password

Task:

1. Write documentation in all three formats.

2. Critically compare the approaches.

3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

```python
def login(user, password, credentials):
    '''
    This function checks if the provided user credentials are valid.
    It takes three parameters: 'user', which is the username; 'password', which is the password; and 'credentials',
    which is a dictionary containing valid username-password pairs. The function returns True if the provided credentials
    match an entry in the credentials dictionary, and False otherwise.
    '''
    return credentials.get(user) == password

def login(user, password, credentials):
    # Check if the provided credentials match an entry in the credentials dictionary
    return credentials.get(user) == password

def login(user, password, credentials):
    ''' Checks if the provided user credentials are valid.
    Parameters:
        user (str): The username to check.
        password (str): The password to check.
        credentials (dict): A dictionary containing valid username-password pairs.
    Returns:
        bool: True if the credentials are valid, False otherwise.
    '''
    return credentials.get(user) == password # Check if the provided credentials match an entry in the credentials dictionary
```

Google style documentation is the suitable one for this because it clearly explains inputs and output, so new developers can quickly learn how the function works The Parameters and Returns sections make the information organized and clear. It follows professional standards, making the code easier to maintain and share.

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.

3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

```python
def add(a, b):
    '''Returns the sum of two numbers.
    Parameters:
    a (int or float): The first number to add.
    b (int or float): The second number to add.
    Returns:   int or float: The sum of a and b.
    Raises:   TypeError: If either a or b is not a number.
    '''
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Both a and b must be numbers.")
    return a + b

def subtract(a, b):
    '''Returns the difference of two numbers.
    Parameters:
    a (int or float): The first number.
    b (int or float): The second number.
    Returns:   int or float: The difference of a and b.
    Raises:   TypeError: If either a or b is not a number.
    '''
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Both a and b must be numbers.")
    return a - b

def multiply(a, b):
    '''Returns the product of two numbers.
    Parameters:
    a (int or float): The first number.
    b (int or float): The second number.
    Returns:   int or float: The product of a and b.
    Raises:   TypeError: If either a or b is not a number.
    '''
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Both a and b must be numbers.")
    return a * b

def divide(a, b):
    '''Returns the quotient of two numbers.
    Parameters:
    a (int or float): The dividend.
    b (int or float): The divisor.
    Returns:   int or float: The quotient of a and b.
    Raises:   TypeError: If either a or b is not a number.
              ZeroDivisionError: If the divisor is zero.
              '''
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Both a and b must be numbers.")

    if b == 0:
        raise ZeroDivisionError("The divisor cannot be zero.")

    return a / b
print(add.__doc__) # returns the docstring of the add function, which describes what the function does and how to use it
print(subtract.__doc__) # returns the docstring of the subtract function, which describes what the function does and how to use it
print(multiply.__doc__) # returns the docstring of the multiply function, which describes what the function does and how to use it
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python
Returns the sum of two numbers.
Parameters:
a (int or float): The first number to add.
b (int or float): The second number to add.
Returns:   int or float: The sum of a and b.
Raises:   TypeError: If either a or b is not a number.

Returns the difference of two numbers.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns:   int or float: The difference of a and b.
Raises:   TypeError: If either a or b is not a number.

Returns the product of two numbers.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns:   int or float: The product of a and b.
Raises:   TypeError: If either a or b is not a number.

Returns the quotient of two numbers.
Parameters:
a (int or float): The dividend.
b (int or float): The divisor.
Returns:   int or float: The quotient of a and b.
Raises:   TypeError: If either a or b is not a number.
        ZeroDivisionError: If the divisor is zero.

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc calculator
Returns the sum of two numbers.
Parameters:
a (int or float): The first number to add.
b (int or float): The second number to add.
Returns:   int or float: The sum of a and b.
Raises:   TypeError: If either a or b is not a number.

Returns the difference of two numbers.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns:   int or float: The difference of a and b.
Raises:   TypeError: If either a or b is not a number.

Returns the product of two numbers.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns:   int or float: The product of a and b.
Raises:   TypeError: If either a or b is not a number.

Returns the quotient of two numbers.
Parameters:
a (int or float): The dividend.
b (int or float): The divisor.
Returns:   int or float: The quotient of a and b.
Raises:   TypeError: If either a or b is not a number.
        ZeroDivisionError: If the divisor is zero.

Help on module calculator:
```

```
    calculator

FUNCTIONS
    add(a, b)
        Returns the sum of two numbers.
        Parameters:
        a (int or float): The first number to add.
        b (int or float): The second number to add.
        Returns:   int or float: The sum of a and b.
        Raises:   TypeError: If either a or b is not a number.

    divide(a, b)
        Returns the quotient of two numbers.
        Parameters:
        a (int or float): The dividend.
        b (int or float): The divisor.
        Returns:   int or float: The quotient of a and b.
        Raises:   TypeError: If either a or b is not a number.
                ZeroDivisionError: If the divisor is zero.

    multiply(a, b)
        Returns the product of two numbers.
        Parameters:
        a (int or float): The first number.
        b (int or float): The second number.
        Returns:   int or float: The product of a and b.
        Raises:   TypeError: If either a or b is not a number.

-- More --
```

index

calculator c:\users\varsh\onedrive\ドキュメント\third\aiac\calculator.py

Functions

add(a, b)

Returns the sum of two numbers.
Parameters:
a (int or float): The first number to add.
b (int or float): The second number to add.
Returns: int or float: The sum of a and b.
Raises: TypeError: If either a or b is not a number.

divide(a, b)

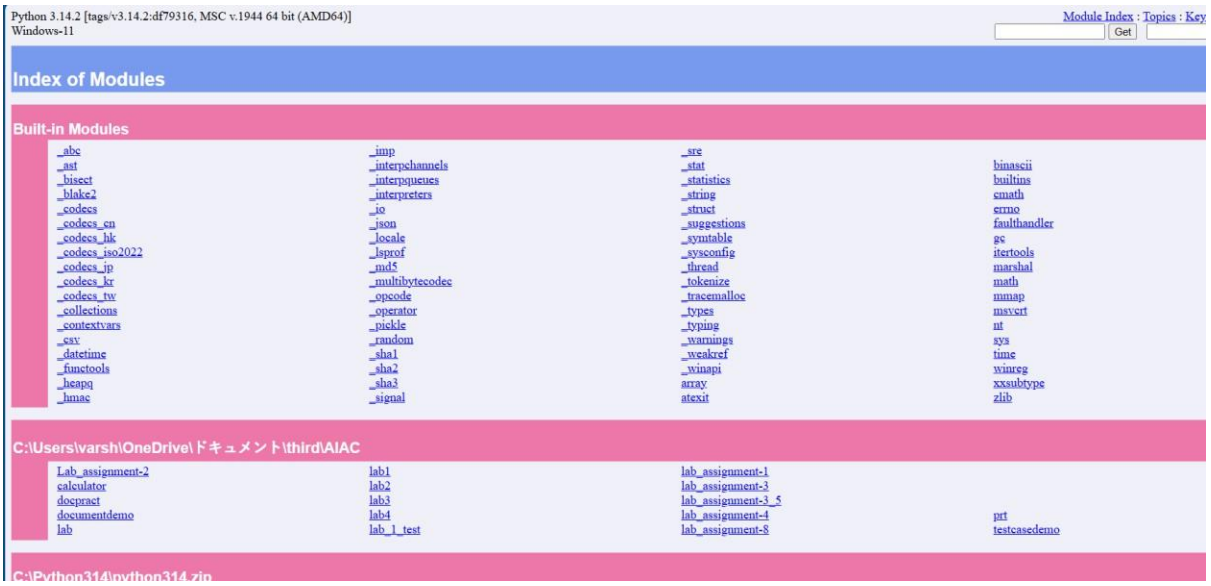
Returns the quotient of two numbers.
Parameters:
a (int or float): The dividend.
b (int or float): The divisor.
Returns: int or float: The quotient of a and b.
Raises: TypeError: If either a or b is not a number.
ZeroDivisionError: If the divisor is zero.

multiply(a, b)

Returns the product of two numbers.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns: int or float: The product of a and b.
Raises: TypeError: If either a or b is not a number.

subtract(a, b)

Returns the difference of two numbers.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns: int or float: The difference of a and b.
Raises: TypeError: If either a or b is not a number.



Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

```python
def decimal_to_binary(n):
    '''Converts a decimal number to its binary representation.
    Parameters:
    n (int): A non-negative integer to be converted to binary.
    Returns:   str: The binary representation of the input decimal number.
    Raises:   ValueError: If n is negative or not an integer.
    '''
    if not isinstance(n, int) or n < 0:
        raise ValueError("Input must be a non-negative integer.")

    if n == 0:
        return "0"

    binary = ""
    while n > 0:
        binary = str(n % 2) + binary
        n //= 2

    return binary

def binary_to_decimal(b):
    '''Converts a binary number (as a string) to its decimal representation.
    Parameters:
    b (str): A string representing a binary number (composed of '0's and '1's).
    Returns:   int: The decimal representation of the input binary number.
    Raises:   ValueError: If b is not a valid binary string.
    '''
    if not isinstance(b, str) or not all(char in '01' for char in b):
        raise ValueError("Input must be a string representing a binary number.")

    decimal = 0
    power = 0

    for digit in reversed(b):
        decimal += int(digit) * (2 ** power)
        power += 1

    return decimal

def decimal_to_hexadecimal(n):
    '''Converts a decimal number to its hexadecimal representation.
    Parameters:
    n (int): A non-negative integer to be converted to hexadecimal.
    Returns:   str: The hexadecimal representation of the input decimal number.
    Raises:   ValueError: If n is negative or not an integer.
    '''
    if not isinstance(n, int) or n < 0:
        raise ValueError("Input must be a non-negative integer.")

    if n == 0:
        return "0"

    hex_digits = "0123456789ABCDEF"
    hexadecimal = ""
```

```
Use help(str) for help on the str class.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc conversion
Help on module conversion:

NAME
    conversion

FUNCTIONS
    binary_to_decimal(b)
        Converts a binary number (as a string) to its decimal representation.
        Parameters:
        b (str): A string representing a binary number (composed of '0's and '1's).
        Returns:   int: The decimal representation of the input binary number.
        Raises:   ValueError: If b is not a valid binary string.

    decimal_to_binary(n)
        Converts a decimal number to its binary representation.
        Parameters:
        n (int): A non-negative integer to be converted to binary.
        Returns:   str: The binary representation of the input decimal number.
        Raises:   ValueError: If n is negative or not an integer.

    decimal_to_hexadecimal(n)
        Converts a decimal number to its hexadecimal representation.
        Parameters:
        n (int): A non-negative integer to be converted to hexadecimal.
        Returns:   str: The hexadecimal representation of the input decimal number.
        Raises:   ValueError: If n is negative or not an integer.

FILE
    c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\conversion.py
```

File C:/Users/varsh/OneDrive/ドキュメント/third/AIAC/conversion.html

index

conversion c:\users\varsh\onedrive\ドキュメント\third\aiac\conversion.py

Functions

binary_to_decimal(b)
Converts a binary number (as a string) to its decimal representation.
Parameters:
b (str): A string representing a binary number (composed of '0's and '1's).
Returns: int: The decimal representation of the input binary number.
Raises: ValueError: If b is not a valid binary string.

decimal_to_binary(n)
Converts a decimal number to its binary representation.
Parameters:
n (int): A non-negative integer to be converted to binary.
Returns: str: The binary representation of the input decimal number.
Raises: ValueError: If n is negative or not an integer.

decimal_to_hexadecimal(n)
Converts a decimal number to its hexadecimal representation.
Parameters:
n (int): A non-negative integer to be converted to hexadecimal.
Returns: str: The hexadecimal representation of the input decimal number.
Raises: ValueError: If n is negative or not an integer.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

```python
def add_course(course_id, name, credits):
    '''
    Adds a course to the course catalog.
    Parameters:
    course_id (str): The unique identifier for the course.
    name (str): The name of the course.
    credits (int): The number of credits for the course.
    Returns:   None
    Raises:   ValueError: If the course_id already exists in the catalog or if credits is not a positive integer.
    '''
    # Implementation code goes here
    if course_id in courses:
        raise ValueError("Course ID already exists in the catalog.")
    if not isinstance(credits, int) or credits <= 0:
        raise ValueError("Credits must be a positive integer.")
    courses[course_id] = {"name": name, "credits": credits}

def remove_course(course_id):
    '''
    Removes a course from the course catalog.
    Parameters:
    course_id (str): The unique identifier for the course to be removed.
    Returns:   None
    Raises:   KeyError: If the course_id does not exist in the catalog.
    '''
    # Implementation code goes here
    if course_id not in courses:
        raise KeyError("Course ID does not exist in the catalog.")
    del courses[course_id]

def get_course(course_id):
    '''
    Retrieves the details of a course from the course catalog.
    Parameters:
    course_id (str): The unique identifier for the course to be retrieved.
    Returns:   dict: A dictionary containing the name and credits of the course.
    Raises:   KeyError: If the course_id does not exist in the catalog.
    '''
    # Implementation code goes here
    if course_id not in courses:
        raise KeyError("Course ID does not exist in the catalog.")
    return courses[course_id]
courses = {} # Initialize an empty dictionary to store course information
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc course
Help on module course:

NAME
    course

FUNCTIONS
    add_course(course_id, name, credits)
        Adds a course to the course catalog.
        Parameters:
        course_id (str): The unique identifier for the course.
        name (str): The name of the course.
        credits (int): The number of credits for the course.
        Returns:   None
        Raises:   ValueError: If the course_id already exists in the catalog or if credits is not a positive integer.

    get_course(course_id)
        Retrieves the details of a course from the course catalog.
        Parameters:
        course_id (str): The unique identifier for the course to be retrieved.
        Returns:   dict: A dictionary containing the name and credits of the course.
        Raises:   KeyError: If the course_id does not exist in the catalog.

    remove_course(course_id)
        Removes a course from the course catalog.
        Parameters:
        course_id (str): The unique identifier for the course to be removed.
        Returns:   None
        Raises:   KeyError: If the course_id does not exist in the catalog.

DATA
    courses = {}

FILE
    c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\course.py
```

File C:/Users/varsh/OneDrive/ドキュメント/third/AIAC/course.html

index

course c:\users\varsh\onedrive\ドキュメント\third\aiac\course.py

Functions

add_course(course_id, name, credits)
Adds a course to the course catalog.
Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.
Returns: None
Raises: ValueError: If the course_id already exists in the catalog or if credits is not a positive integer.

get_course(course_id)
Retrieves the details of a course from the course catalog.
Parameters:
course_id (str): The unique identifier for the course to be retrieved.
Returns: dict: A dictionary containing the name and credits of the course.
Raises: KeyError: If the course_id does not exist in the catalog.

remove_course(course_id)
Removes a course from the course catalog.
Parameters:
course_id (str): The unique identifier for the course to be removed.
Returns: None
Raises: KeyError: If the course_id does not exist in the catalog.

Data

courses = {}